DBMS Project Report

Submitted for

DATABASE MANAGEMENT SYSTEM (UCS310)

PROJECT TITLE: CampusPay – RFID-Based Unified Tap & Pay System for College Campus

Submitted by:

Ikansh Mahajan (102303754)
Prisha Aggarwal (102303758)
Parth Biyani (102303759)

Submitted to:

Dr. Swati Sharma



Computer Science and Engineering Department

Thapar Institute of Engineering & Technology, Patiala

Jan-June 2025

Table of Contents

Introduction 2
Problem Statement 3
ER Diagram 4
ER to Table Mapping 5
Normalized Tables 6
Sample Database 7
SQL Queries 9
Query to find the transaction between 4pm and 7pm grouped by scanner: 9
Query to display the users with balance lower than ₹900: 9
PL/SQL Functions and Procedures and their executions 10
get_balance() function: 10
update_wallet_active_status procedure: 11
process_topup procedure: 13
process_transaction procedure: 15
process_p2p_payment procedure: 17
Conclusion 21
References 22

Introduction

The "CampusPay" system introduces a ground-breaking RFID-based unified Tap-and-Pay system tailored for Thapar Institute of Engineering and Technology (TIET), harnessing the existing RFID-enabled ID cards issued to students, faculty, and staff. This project seeks to unify the fragmented payment ecosystem across the TIET campus, replacing the myriad of apps and payment methods currently used at canteens, libraries, stationery shops, and hostels with a single, efficient RFID-based unified Tap-and-Pay system. By deploying custom RFID scanners linked to a centralized SQL database, CampusPay facilitates fast, secure, and reliable transactions, reducing reliance on internet connectivity at point-of-sale (POS) terminals. The system is supported by a robust backend featuring tables for users, wallets, scanners, transactions, and top-ups, managed through PL/SQL procedures that handle payments, top-ups, peer-to-peer transfers, and wallet status updates.

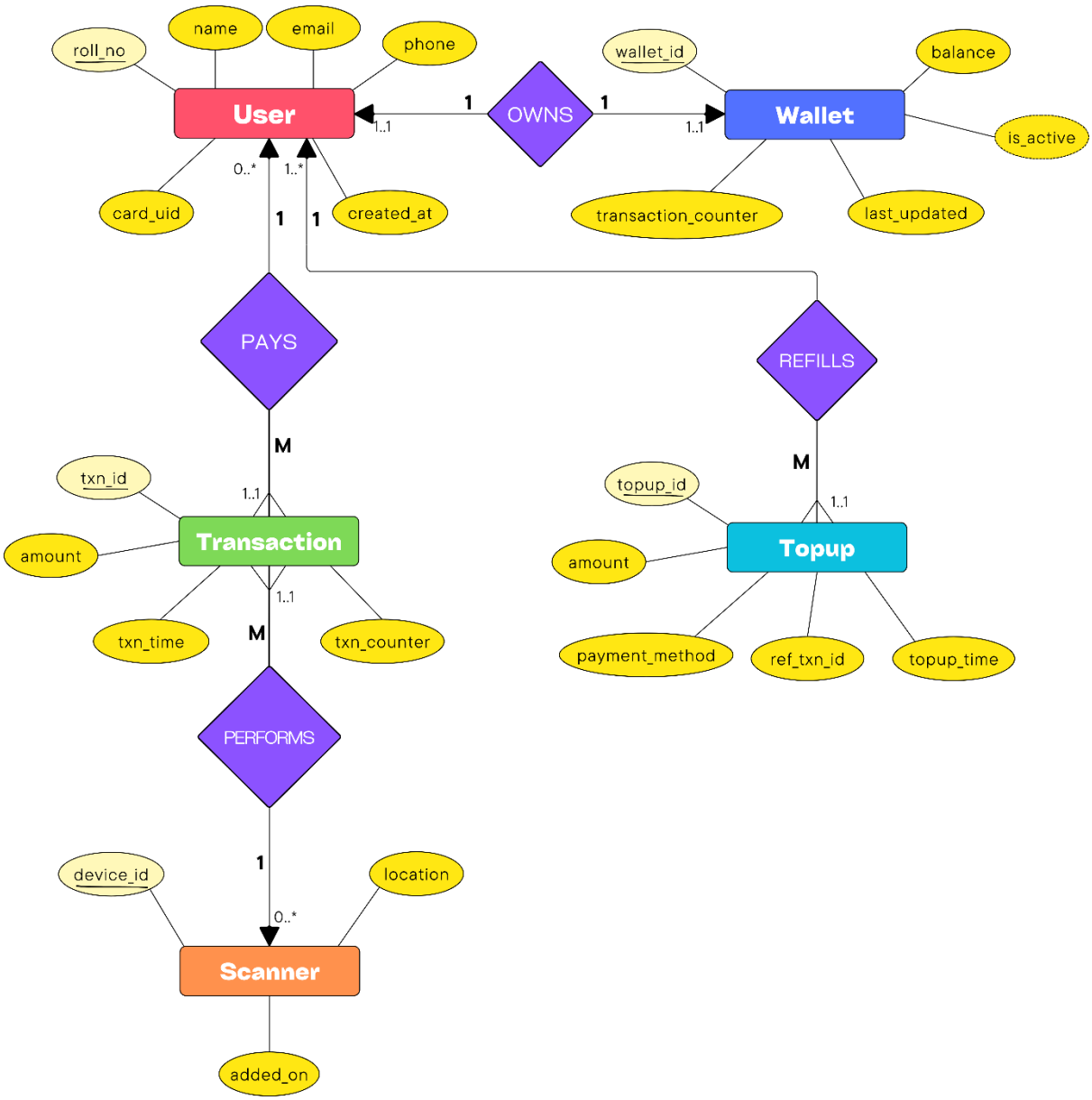
CampusPay enhances user convenience while promoting sustainability by minimizing the need for multiple digital payment platforms, thus lowering the associated digital footprint. For vendors, the RFID-based unified Tap-and-Pay system simplifies reconciliation and settlement, ensuring accurate and timely financial reporting. The system incorporates security features, such as wallet deactivation via the “is_active” flag and centralized transaction logging, to prevent fraud and maintain data integrity. Furthermore, its scalable architecture opens opportunities for future enhancements, such as advanced analytics for spending trends and integration with campus resource management tools. Through this initiative, TIET positions itself as a leader in technological innovation, fostering a cashless, efficient, and inclusive campus environment that aligns with the institute’s vision of embracing smart solutions by a payment method made by the students of the college.

Problem Statement

At Thapar Institute of Engineering and Technology, the current payment landscape suffers from a lack of uniformity, with students and staff relying on various apps and methods—such as UPI apps (Google Pay, PhonePe), vendor-specific platforms, and cash—to conduct transactions across campus facilities. This fragmentation results in an inconsistent user experience, where individuals must navigate multiple platforms, often facing delays due to internet issues or app-specific limitations. For vendors, managing diverse payment systems complicates end-of-day reconciliation, delays settlements, and increases the likelihood of errors in financial reporting. Moreover, the absence of a centralized system limits the administration's ability to monitor transactions, analyze spending patterns, or ensure financial oversight effectively.

The lack of a unified payment solution also raises concerns about scalability and security as the campus expands. Handling multiple payment methods becomes increasingly unsustainable, and the absence of centralized control heightens the risk of fraudulent activities. The "CampusPay" RFID-based unified Tap-and-Pay system addresses these challenges by implementing a solution that leverages existing ID cards. Integrated with low-cost RFID scanners and a secure SQL backend, it ensures seamless transactions, reduces operational overhead for vendors, and provides a scalable platform for future growth. This initiative aims to create a cohesive, secure, and efficient RFID-based unified Tap-and-Pay system that enhances the campus experience for all stakeholders.

ER Diagram


roll_no
name
email
phone
User
1
OWNS
1
wallet_id
balance
Wallet
is_active
1..1
1..1
0..*
1..*
card_uid
created_at
1
1
transaction_counter
last_updated
PAYS
REFILLS
M
M
txn_id
1..1
topup_id
1..1
amount
Transaction
amount
Topup
1..1
txn_time
M
txn_counter
payment_method
ref_txn_id
topup_time
PERFORMS
1
device_id
location
0..*
Scanner
added_on

ER to Table Mapping

Entity	Table Name	Attributes
User	users	Roll_no, name, email, phone, card_uid, created_at
Wallet	wallets	wallet_id, roll_no, balance, transaction_counter, last_updated
Transaction	transactions	txn_id, roll_no, amount, location, txn_time, txn_counter, device_id
Topup	topups	topup_id, roll_no, amount, payment_method, rref_txn_id, topup_time
Scanner	scanners	device_id, location, added_on

In mapping the conceptual ER Diagram to relational tables, we define primary keys and foreign keys and impose cardinalities: the one-to-one relationship of User–Wallet is facilitated by a UNIQUE constraint on wallets.roll_no, and the one-to-many User–Transactions and User–Topups relationships are based on non-null foreign keys with ON DELETE RESTRICT for historical integrity. Check constraints on topup and transaction values ensure they are positive, and default values (wallets.transaction_counter default 0) ensure initialization. Folding relationship entities like PAYS and REFILLS into transactions and topups tables directly, we eliminate unnecessary join tables, achieving third-normal-form—eliminating multi-valued attributes and update anomalies—without sacrificing auditability through immutable timestamps.

Normalized Tables

All five tables conform to the Third Normal Form: each has a primary key, attributes are atomic, and no non-key attribute depends on another non-key attribute. Foreign keys link related tables without redundancy.

1. users

- Primary Key: *roll_no*
- Atomic fields: name, email, phone, card_uid, created_at
- No repeating groups or transitive dependencies.

2. wallets

- Primary Key: *wallet_id*
- Foreign Key: *roll_no → users*
- Attributes balance, transaction_counter, last_updated describe only the wallet.
- One-to-one link via roll_no ensures no redundant user data.

3. transactions

- Primary Key: *txn_id*
- Foreign Key: *roll_no → users, device_id → scanners*
- Stores only event details (amount, txn_time, location, txn_counter).
- No duplication of user or scanner attributes.

4. topups

- Primary Key*: topup_id*
- Foreign Key: *roll_no → users*
- Atomic attributes: amount, payment_method, ref_txn_id, topup_time.
- Each refill is logged independently, with no redundant balance data.

5. scanners

- Primary Key: *device_id*
- Atomic attributes: location, added_on.
- No multi-valued or derived fields; each scanner is uniquely identified.

Sample Database

Users table:

	ROLL_NO	NAME	EMAIL	PHONE	CARD_UID	CREATED_AT
1	1	Aarav Mehta	aarav@thapar.edu	9876543210	04A1B2C3	7/20/2023, 12:00:00 AM
2	2	Sara Nair	sara@thapar.edu	9823456710	04D3E2F1	7/21/2023, 12:00:00 AM
3	3	Ravi Sharma	ravi@thapar.edu	9812345671	04FFEE11	7/21/2023, 12:00:00 AM
4	4	Meena Rao	meena@thapar.edu	9898989898	0499AABB	7/21/2023, 12:00:00 AM
5	5	Karan Joshi	karan@thapar.edu	9723232323	04CCDDEE	7/22/2023, 12:00:00 AM
6	6	Sneha Das	sneha@thapar.edu	9765432101	04224466	7/22/2023, 12:00:00 AM
7	7	Anil Kapoor	anil@thapar.edu	9834343434	0488AA11	7/22/2023, 12:00:00 AM
8	8	Pooja Singh	pooja@thapar.edu	9988776655	046611AA	7/23/2023, 12:00:00 AM
9	9	Vikram Verma	vikram@thapar.edu	9888777666	0433BBDD	7/18/2024, 12:00:00 AM
10	10	Divya Iyer	divya@thapar.edu	9777888999	0455CCEE	7/19/2024, 12:00:00 AM

Wallets table:

	WALLET_ID	ROLL_NO	BALANCE	TRANSACTION_COUNTER	LAST_UPDATED	IS_ACTIVE
1	101	1	892	2	2025-04-20T14:10:32Z	1
2	102	2	1395	3	2025-04-23T14:11:34Z	1
3	103	3	980	2	2025-04-21T12:38:40Z	1
4	104	4	929	4	2025-04-23T11:37:00Z	1
5	105	5	785	2	2025-04-21T16:05:26Z	1
6	106	6	810	4	2025-04-25T16:30:50Z	1
7	107	7	1040	4	2025-04-24T22:38:39Z	1
8	108	8	925	2	2025-04-22T19:55:01Z	1
9	109	9	905	3	2025-04-25T08:32:41Z	1
10	110	10	920	3	2025-04-25T17:19:39Z	1

Scanners table:

	DEVICE_ID	LOCATION	ADDED_ON
1	scanner_001	Kravings - Dominoes	4/20/2025, 12:00:00 AM
2	scanner_002	Aahar	4/20/2025, 12:00:00 AM
3	scanner_003	TSLAS Canteen	4/20/2025, 12:00:00 AM
4	scanner_004	Library	4/20/2025, 12:00:00 AM
5	scanner_005	Just Foodz	4/20/2025, 12:00:00 AM
6	scanner_006	Bombay Munchery	4/20/2025, 12:00:00 AM
7	scanner_007	Shadowz Saloon	4/20/2025, 12:00:00 AM
8	scanner_008	Hostel C Mess	4/20/2025, 12:00:00 AM
9	scanner_009	Waterfront Cafe	4/20/2025, 12:00:00 AM
10	scanner_010	Jaggi	4/20/2025, 12:00:00 AM
11	P2P_TRANSFER	Peer-to-Peer Transfer	4/20/2025, 12:00:00 AM

Transactions table:

	TXN_ID	ROLL_NO	AMOUNT	TXN_TIME	TXN_COUNTER	DEVICE_ID
1	201	1	108	2025-04-20T14:10:32Z	1	scanner_001
2	202	2	130	2025-04-20T18:24:55Z	1	scanner_002
3	203	3	20	2025-04-21T12:38:40Z	1	scanner_003
4	204	4	13	2025-04-21T09:16:48Z	1	scanner_004
5	205	5	215	2025-04-21T16:05:26Z	1	scanner_005
6	206	6	70	2025-04-22T15:59:17Z	1	scanner_006
7	207	7	200	2025-04-22T13:43:59Z	1	scanner_007
8	208	8	75	2025-04-22T19:55:01Z	1	scanner_008
9	209	9	30	2025-04-23T10:10:34Z	1	scanner_009
10	210	10	35	2025-04-23T17:23:12Z	1	scanner_010
11	211	4	158	2025-04-23T11:37:00Z	2	scanner_001
12	212	6	105	2025-04-24T20:46:51Z	2	scanner_002
13	213	7	60	2025-04-24T15:37:28Z	2	scanner_010
14	214	10	45	2025-04-25T17:19:39Z	2	scanner_010
15	215	9	65	2025-04-25T08:32:41Z	2	scanner_008
16	216	6	15	2025-04-25T16:30:50Z	3	scanner_009

Topups table:

	TOPUP_ID	ROLL_NO	AMOUNT	PAYMENT_METHOD	REF_TXN_ID	TOPUP_TIME
1	301	1	1000	INITIAL DEPOSIT	301	2025-04-20T00:00:00Z
2	302	2	1000	INITIAL DEPOSIT	302	2025-04-20T00:00:00Z
3	303	3	1000	INITIAL DEPOSIT	303	2025-04-20T00:00:00Z
4	304	4	1000	INITIAL DEPOSIT	304	2025-04-20T00:00:00Z
5	305	5	1000	INITIAL DEPOSIT	305	2025-04-20T00:00:00Z
6	306	6	1000	INITIAL DEPOSIT	306	2025-04-20T00:00:00Z
7	307	7	1000	INITIAL DEPOSIT	307	2025-04-20T00:00:00Z
8	308	8	1000	INITIAL DEPOSIT	308	2025-04-20T00:00:00Z
9	309	9	1000	INITIAL DEPOSIT	309	2025-04-20T00:00:00Z
10	310	10	1000	INITIAL DEPOSIT	310	2025-04-20T00:00:00Z
11	311	4	100	GPay	upi123abc	2025-04-22T16:56:27Z
12	312	2	525	Debit Card	dc456def	2025-04-23T14:11:34Z
13	313	7	300	Net Banking	neft789ghi	2025-04-24T22:38:39Z

SQL Queries

Query to find the transaction between 4pm and 7pm grouped by scanner:

```
-- Find the transactions between 4pm and 7pm everyday grouped by scanner
SELECT
    t.device_id,
    s.location,
    COUNT(t.txn_id) AS total_transactions,
    SUM(t.amount) AS total_amount
FROM transactions t
JOIN scanners s ON t.device_id = s.device_id
WHERE EXTRACT(HOUR FROM t.txn_time) BETWEEN 16 AND 18
GROUP BY t.device_id, s.location
ORDER BY total_transactions DESC;
```

Output:

	DEVICE_ID	LOCATION	TOTAL_TRANSACTIONS	TOTAL_AMOUNT
1	scanner_010	Jaggi	2	80
2	scanner_009	Waterfront Cafe	1	15
3	scanner_002	Aahar	1	130
4	scanner_005	Just Foodz	1	215

Query to display the users with balance lower than ₹900:

```
-- Display the users with balance lower than ₹900
SELECT
    u.roll_no,
    u.name,
    w.balance
FROM users u
JOIN wallets w ON u.roll_no = w.roll_no
WHERE w.balance < 900
ORDER BY w.balance ASC;
```

Output:

	ROLL_NO	NAME	BALANCE
1	5	Karan Joshi	785
2	6	Sneha Das	810
3	1	Aarav Mehta	892

PL/SQL Functions and Procedures and their executions

get_balance() function:

```
-- PL/SQL Function that will return the balance of the asked roll_no
CREATE OR REPLACE FUNCTION get_balance(p_roll_no IN NUMBER)
RETURN NUMBER
IS
    v_balance NUMBER;
BEGIN
    SELECT balance
    INTO v_balance
    FROM wallets
    WHERE roll_no = p_roll_no;

    RETURN v_balance;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END;
/
```

Testing the function:

```
-- Testing the PL/SQL function that returns the current balance of the given roll_no
SELECT get_balance(1) FROM dual;
```

Output:

	GET_BALANCE(1)
1	892

update_wallet_active_status procedure:

```
-- PL/SQL Procedure to update wallet active status
CREATE OR REPLACE PROCEDURE update_wallet_active_status(
    p_roll_no IN NUMBER,
    p_is_active IN NUMBER
)
IS
    v_wallet_id NUMBER;
BEGIN
    -- Validate input
    IF p_is_active NOT IN (0, 1) THEN
        DBMS_OUTPUT.PUT_LINE('Warning: is_active must be 0 or 1. Skipping update.');
        RETURN;
    END IF;

    -- Check if wallet exists
    BEGIN
        SELECT wallet_id
        INTO v_wallet_id
        FROM wallets
        WHERE roll_no = p_roll_no;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for roll number ' || p_roll_no || '. Skipping update.');
            RETURN;
    END;

    -- Update the is_active flag
    UPDATE wallets
    SET is_active = p_is_active,
        last_updated = SYSTIMESTAMP
    WHERE roll_no = p_roll_no;

    -- Commit the transaction
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Wallet active status updated for roll number ' || p_roll_no ||
                         ' (is_active: ' || p_is_active || ').');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error updating wallet active status for roll number ' || p_roll_no || ': ' || SQLERRM);
END;
/
```

Testing the procedure:

```
-- Testing the update_wallet_active_status procedure
BEGIN
    -- Test Update (Activate Wallet)
    DBMS_OUTPUT.PUT_LINE('Testing activation of wallet for roll number 1...');
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 0
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test Update (Deactivate Wallet)
    DBMS_OUTPUT.PUT_LINE('Testing deactivation of wallet for roll number 1...');
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 1
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test Update (Invalid Value)
    DBMS_OUTPUT.PUT_LINE('Testing update with invalid value...');
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 2
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test Update (Non-existent Roll Number)
    DBMS_OUTPUT.PUT_LINE('Testing update with invalid roll number...');
    update_wallet_active_status(
        p_roll_no => 999,
        p_is_active => 0
    );
END;
/
```

Output:

```
Testing activation of wallet for roll number 1...
Wallet active status updated for roll number 1 (is_active: 0).


Testing deactivation of wallet for roll number 1...
Wallet active status updated for roll number 1 (is_active: 1).


Testing update with invalid value...
Warning: is_active must be 0 or 1. Skipping update.


Testing update with invalid roll number...
Warning: No wallet found for roll number 999. Skipping update.


PL/SQL procedure successfully completed.
```

process_topup procedure:

```
-- PL/SQL Procedure to process wallet balance topups
CREATE OR REPLACE PROCEDURE process_topup(
    p_roll_no IN NUMBER,
    p_amount IN NUMBER,
    p_payment_method IN VARCHAR2,
    p_ref_txn_id IN VARCHAR2
)
IS
    v_wallet_id NUMBER;
    v_current_balance NUMBER;
    v_next_topup_id NUMBER;
    v_updated_balance NUMBER;
BEGIN
    -- Validate inputs
    IF p_amount <= 0 THEN
        DBMS_OUTPUT.PUT_LINE('Warning: Top-up amount must be positive. Skipping top-up.');
        RETURN;
    END IF;
    IF p_ref_txn_id IS NULL THEN
        DBMS_OUTPUT.PUT_LINE('Warning: Reference transaction ID cannot be null. Skipping top-up.');
        RETURN;
    END IF;

    -- Check if wallet is active
    DECLARE
        v_is_active NUMBER;
    BEGIN
        SELECT is_active
        INTO v_is_active
        FROM wallets
        WHERE roll_no = p_roll_no;
        IF v_is_active = 0 THEN
            DBMS_OUTPUT.PUT_LINE('Warning: Wallet for roll number ' || p_roll_no || ' is inactive. Skipping top-up.');
            RETURN;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for roll number ' || p_roll_no || '. Skipping top-up.');
            RETURN;
    END;

    -- Get wallet details
    BEGIN
        SELECT wallet_id, balance
        INTO v_wallet_id, v_current_balance
        FROM wallets
        WHERE roll_no = p_roll_no;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for roll number ' || p_roll_no || '. Skipping top-up.');
            RETURN;
    END;

    -- Determine next topup_id
    SELECT NVL(MAX(topup_id), 0) + 1
    INTO v_next_topup_id
    FROM topups;

    -- Insert top-up record
    INSERT INTO topups (topup_id, roll_no, amount, payment_method, ref_txn_id, topup_time)
    VALUES (v_next_topup_id, p_roll_no, p_amount, p_payment_method, p_ref_txn_id, SYSTIMESTAMP);

    -- Update wallet balance
    UPDATE wallets
    SET balance = v_current_balance + p_amount,
        transaction_counter = transaction_counter + 1,
        last_updated = SYSTIMESTAMP
    WHERE wallet_id = v_wallet_id;

    -- Commit the transaction
    COMMIT;

    -- Fetch the updated balance
    SELECT balance
    INTO v_updated_balance
    FROM wallets
    WHERE roll_no = p_roll_no;

    -- Output the success message with updated balance
    DBMS_OUTPUT.PUT_LINE('Top-up successful for roll number ' || p_roll_no ||
                         ' (new balance: ' || v_updated_balance || ').');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error during top-up for roll number ' || p_roll_no || ': ' || SQLERRM);
END;
/
```

Testing the procedure:

```
-- Testing the process_topup procedure
BEGIN
    -- Test Top-Up (Successful Case)
    DBMS_OUTPUT.PUT_LINE('Testing top-up for roll number 1...');
    process_topup(
        p_roll_no => 1,
        p_amount => 500.00,
        p_payment_method => 'Credit Card',
        p_ref_txn_id => 'cc789abc'
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test Top-Up (Invalid Amount)
    DBMS_OUTPUT.PUT_LINE('Testing top-up with invalid amount...');
    process_topup(
        p_roll_no => 1,
        p_amount => -100.00,
        p_payment_method => 'Credit Card',
        p_ref_txn_id => 'cc123abc'
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test Top-Up (Inactive Wallet)
    DBMS_OUTPUT.PUT_LINE('Testing top-up with inactive wallet...');
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 0
    );
    process_topup(
        p_roll_no => 1,
        p_amount => 200.00,
        p_payment_method => 'Credit Card',
        p_ref_txn_id => 'cc456def'
    );
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 1
    );
END;
/
```

Output:

```
Testing top-up for roll number 1...
Top-up successful for roll number 1 (new balance: 1392).


Testing top-up with invalid amount...
Warning: Top-up amount must be positive. Skipping top-up.


Testing top-up with inactive wallet...
Wallet active status updated for roll number 1 (is_active: 0).
Warning: Wallet for roll number 1 is inactive. Skipping top-up.
Wallet active status updated for roll number 1 (is_active: 1).


PL/SQL procedure successfully completed.
```

process_transaction procedure:

```
-- PL/SQL Procedure to process transactions
CREATE OR REPLACE PROCEDURE process_transaction(
    p_roll_no IN NUMBER,
    p_amount IN NUMBER,
    p_device_id IN VARCHAR2
)
IS
    v_wallet_id NUMBER;
    v_current_balance NUMBER;
    v_next_txn_id NUMBER;
    v_updated_balance NUMBER;
BEGIN
    -- Validate inputs
    IF p_amount <= 0 THEN
        DBMS_OUTPUT.PUT_LINE('Warning: Transaction amount must be positive. Skipping transaction.');
        RETURN;
    END IF;
    IF p_device_id IS NULL THEN
        DBMS_OUTPUT.PUT_LINE('Warning: Device ID cannot be null. Skipping transaction.');
        RETURN;
    END IF;

    -- Check if wallet is active
    DECLARE
        v_is_active NUMBER;
    BEGIN
        SELECT is_active
        INTO v_is_active
        FROM wallets
        WHERE roll_no = p_roll_no;
        IF v_is_active = 0 THEN
            DBMS_OUTPUT.PUT_LINE('Warning: Wallet for roll number ' || p_roll_no || ' is inactive. Skipping transaction.');
            RETURN;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for roll number ' || p_roll_no || '. Skipping transaction.');
            RETURN;
    END;

    -- Get wallet details
    BEGIN
        SELECT wallet_id, balance
        INTO v_wallet_id, v_current_balance
        FROM wallets
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for roll number ' || p_roll_no || '. Skipping transaction.');
            RETURN;
    END;

    -- Check sufficient balance
    IF v_current_balance < p_amount THEN
        DBMS_OUTPUT.PUT_LINE('Warning: Insufficient balance for roll number ' || p_roll_no || '. Skipping transaction.');
        RETURN;
    END IF;

    -- Determine next txn_id
    SELECT NVL(MAX(txn_id), 0) + 1
    INTO v_next_txn_id
    FROM transactions;

    -- Insert transaction record
    INSERT INTO transactions (txn_id, roll_no, amount, txn_time, txn_counter, device_id)
    VALUES (v_next_txn_id, p_roll_no, p_amount, SYSTIMESTAMP,
            (SELECT NVL(MAX(txn_counter), 0) + 1 FROM transactions WHERE roll_no = p_roll_no),
            p_device_id);

    -- Update wallet balance
    UPDATE wallets
    SET balance = v_current_balance - p_amount,
        transaction_counter = transaction_counter + 1,
        last_updated = SYSTIMESTAMP
    WHERE wallet_id = v_wallet_id;

    -- Commit the transaction
    COMMIT;

    -- Fetch the updated balance
    SELECT balance
    INTO v_updated_balance
    FROM wallets
    WHERE roll_no = p_roll_no;

    -- Output the success message with updated balance
    DBMS_OUTPUT.PUT_LINE('Transaction successful for roll number ' || p_roll_no ||
                         ' (new balance: ' || v_updated_balance || ').');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error during transaction for roll number ' || p_roll_no || ': ' || SQLERRM);
END;
/
```

Testing the procedure:

```
-- Testing the process_transaction procedure
BEGIN
    -- Test Transaction (Successful Case)
    DBMS_OUTPUT.PUT_LINE('Testing transaction for roll number 1...');
    process_transaction(
        p_roll_no => 1,
        p_amount => 200.00,
        p_device_id => 'scanner_001'
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test Transaction (Insufficient Balance)
    DBMS_OUTPUT.PUT_LINE('Testing transaction with insufficient balance...');
    process_transaction(
        p_roll_no => 1,
        p_amount => 5000.00, -- More than current balance
        p_device_id => 'scanner_001'
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test Transaction (Non-existent Roll Number)
    DBMS_OUTPUT.PUT_LINE('Testing transaction with invalid roll number...');
    process_transaction(
        p_roll_no => 999, -- Non-existent roll_no
        p_amount => 100.00,
        p_device_id => 'scanner_001'
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test Transaction (Inactive Wallet)
    DBMS_OUTPUT.PUT_LINE('Testing transaction with inactive wallet...');
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 0
    );
    process_transaction(
        p_roll_no => 1,
        p_amount => 100.00,
        p_device_id => 'scanner_001'
    );
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 1
    );
END;
/
```

Output:

```
Testing transaction for roll number 1...
Transaction successful for roll number 1 (new balance: 1192).


Testing transaction with insufficient balance...
Warning: Insufficient balance for roll number 1. Skipping transaction.


Testing transaction with invalid roll number...
Warning: No wallet found for roll number 999. Skipping transaction.


Testing transaction with inactive wallet...
Wallet active status updated for roll number 1 (is_active: 0).
Warning: Wallet for roll number 1 is inactive. Skipping transaction.
Wallet active status updated for roll number 1 (is_active: 1).


PL/SQL procedure successfully completed.
```

process_p2p_payment procedure:

```
-- PL/SQL procedure to handle Peer-to-Peer (P2P) payments
CREATE OR REPLACE PROCEDURE process_p2p_payment(
    p_sender_roll_no IN NUMBER,
    p_receiver_roll_no IN NUMBER,
    p_amount IN NUMBER
)
IS
    v_sender_wallet_id NUMBER;
    v_sender_balance NUMBER;
    v_receiver_wallet_id NUMBER;
    v_receiver_balance NUMBER;
    v_next_txn_id NUMBER;
    v_updated_sender_balance NUMBER;
    v_updated_receiver_balance NUMBER;
BEGIN
    -- Validate inputs
    IF p_amount <= 0 THEN
        DBMS_OUTPUT.PUT_LINE('Warning: Payment amount must be positive. Skipping P2P payment.');
        RETURN;
    END IF;
    IF p_sender_roll_no = p_receiver_roll_no THEN
        DBMS_OUTPUT.PUT_LINE('Warning: Sender and receiver cannot be the same. Skipping P2P payment.');
        RETURN;
    END IF;

    -- Check if sender wallet is active
    DECLARE
        v_is_active NUMBER;
    BEGIN
        SELECT is_active
        INTO v_is_active
        FROM wallets
        WHERE roll_no = p_sender_roll_no;
        IF v_is_active = 0 THEN
            DBMS_OUTPUT.PUT_LINE('Warning: Wallet for sender roll number ' || p_sender_roll_no || ' is inactive. Skipping P2P payment.');
            RETURN;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for sender roll number ' || p_sender_roll_no || '. Skipping P2P payment.');
            RETURN;
    END;

    -- Check if receiver wallet is active
    DECLARE
        v_is_active NUMBER;
    BEGIN
        SELECT is_active
        INTO v_is_active
        FROM wallets
        WHERE roll_no = p_receiver_roll_no;
        IF v_is_active = 0 THEN
            DBMS_OUTPUT.PUT_LINE('Warning: Wallet for receiver roll number ' || p_receiver_roll_no || ' is inactive. Skipping P2P payment.');
            RETURN;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for receiver roll number ' || p_receiver_roll_no || '. Skipping P2P payment.');
            RETURN;
    END;

    -- Get sender wallet details
    BEGIN
        SELECT wallet_id, balance
        INTO v_sender_wallet_id, v_sender_balance
        FROM wallets
        WHERE roll_no = p_sender_roll_no;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for sender roll number ' || p_sender_roll_no || '. Skipping P2P payment.');
            RETURN;
    END;

    -- Get receiver wallet details
    BEGIN
        SELECT wallet_id, balance
        INTO v_receiver_wallet_id, v_receiver_balance
        FROM wallets
        WHERE roll_no = p_receiver_roll_no;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Warning: No wallet found for receiver roll number ' || p_receiver_roll_no || '. Skipping P2P payment.');
            RETURN;
    END;

    -- Check sufficient balance
    IF v_sender_balance < p_amount THEN
        DBMS_OUTPUT.PUT_LINE('Warning: Insufficient balance for sender roll number ' || p_sender_roll_no || '. Skipping P2P payment.');
        RETURN;
    END IF;
```

```
    -- Determine next transaction ID for sender
    SELECT NVL(MAX(txn_id), 0) + 1
    INTO v_next_txn_id
    FROM transactions;

    -- Record transaction for sender (deduction)
    INSERT INTO transactions (txn_id, roll_no, amount, txn_time, txn_counter, device_id)
    VALUES (v_next_txn_id, p_sender_roll_no, p_amount, SYSTIMESTAMP,
            (SELECT NVL(MAX(txn_counter), 0) + 1 FROM transactions WHERE roll_no = p_sender_roll_no),
            'P2P_TRANSFER');

    -- Update sender wallet
    UPDATE wallets
    SET balance = balance - p_amount,
        transaction_counter = transaction_counter + 1,
        last_updated = SYSTIMESTAMP
    WHERE wallet_id = v_sender_wallet_id;

    -- Determine next transaction ID for receiver
    SELECT NVL(MAX(txn_id), 0) + 1
    INTO v_next_txn_id
    FROM transactions;

    -- Record transaction for receiver (addition)
    INSERT INTO transactions (txn_id, roll_no, amount, txn_time, txn_counter, device_id)
    VALUES (v_next_txn_id, p_receiver_roll_no, p_amount, SYSTIMESTAMP,
            (SELECT NVL(MAX(txn_counter), 0) + 1 FROM transactions WHERE roll_no = p_receiver_roll_no),
            'P2P_TRANSFER');

    -- Update receiver wallet
    UPDATE wallets
    SET balance = balance + p_amount,
        transaction_counter = transaction_counter + 1,
        last_updated = SYSTIMESTAMP
    WHERE wallet_id = v_receiver_wallet_id;

    -- Commit the transaction
    COMMIT;

    -- Fetch the updated balances for sender and receiver
    SELECT balance
    INTO v_updated_sender_balance
    FROM wallets
    WHERE roll_no = p_sender_roll_no;

    SELECT balance
    INTO v_updated_receiver_balance
    FROM wallets
    WHERE roll_no = p_receiver_roll_no;

    -- Output the success message with updated balances
    DBMS_OUTPUT.PUT_LINE('P2P payment successful: ' || p_amount || ' transferred from roll number ' || p_sender_roll_no ||
                         ' (new balance: ' || v_updated_sender_balance || ') to roll number ' || p_receiver_roll_no ||
                         ' (new balance: ' || v_updated_receiver_balance || ').');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error during P2P payment from roll number ' || p_sender_roll_no || ' to ' || p_receiver_roll_no || ': ' || SQLERRM);
END;
/
```

Testing the procedure:

```
-- Testing the process_p2p_payment procedure
BEGIN
    -- Test P2P Payment (Successful Case)
    DBMS_OUTPUT.PUT_LINE('Testing P2P payment from roll number 1 to 2...');
    DBMS_OUTPUT.PUT_LINE('');
    process_p2p_payment(
        p_sender_roll_no => 1,
        p_receiver_roll_no => 2,
        p_amount => 300.00
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test P2P Payment (Insufficient Balance)
    DBMS_OUTPUT.PUT_LINE('Testing P2P payment with insufficient balance...');
    process_p2p_payment(
        p_sender_roll_no => 1,
        p_receiver_roll_no => 2,
        p_amount => 2000.00 -- More than current balance of 892.00
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test P2P Payment (Same Sender and Receiver)
    DBMS_OUTPUT.PUT_LINE('Testing P2P payment with same sender and receiver...');
    process_p2p_payment(
        p_sender_roll_no => 1,
        p_receiver_roll_no => 1,
        p_amount => 100.00
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test P2P Payment (Non-existent Receiver)
    DBMS_OUTPUT.PUT_LINE('Testing P2P payment with invalid receiver...');
    process_p2p_payment(
        p_sender_roll_no => 1,
        p_receiver_roll_no => 999, -- Non-existent roll_no
        p_amount => 100.00
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test P2P Payment (Inactive Sender Wallet)
    DBMS_OUTPUT.PUT_LINE('Testing P2P payment with inactive sender wallet...');
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 0
    );
    process_p2p_payment(
        p_sender_roll_no => 1,
        p_receiver_roll_no => 2,
        p_amount => 100.00
    );
    update_wallet_active_status(
        p_roll_no => 1,
        p_is_active => 1
    );

    DBMS_OUTPUT.PUT_LINE('');
    DBMS_OUTPUT.PUT_LINE('');

    -- Test P2P Payment (Inactive Receiver Wallet)
    DBMS_OUTPUT.PUT_LINE('Testing P2P payment with inactive receiver wallet...');
    update_wallet_active_status(
        p_roll_no => 2,
        p_is_active => 0
    );
    process_p2p_payment(
        p_sender_roll_no => 1,
        p_receiver_roll_no => 2,
        p_amount => 100.00
    );
    update_wallet_active_status(
        p_roll_no => 2,
        p_is_active => 1
    );
END;
/
```

Output:

```
Testing P2P payment from roll number 1 to 2...

P2P payment successful: 300 transferred from roll number 1 (new balance: 892) to
roll number 2 (new balance: 1695).


Testing P2P payment with insufficient balance...
Warning: Insufficient balance for sender roll number 1. Skipping P2P payment.


Testing P2P payment with same sender and receiver...
Warning: Sender and receiver cannot be the same. Skipping P2P payment.


Testing P2P payment with invalid receiver...
Warning: No wallet found for receiver roll number 999. Skipping P2P payment.


Testing P2P payment with inactive sender wallet...
Wallet active status updated for roll number 1 (is_active: 0).
Warning: Wallet for sender roll number 1 is inactive. Skipping P2P payment.
Wallet active status updated for roll number 1 (is_active: 1).


Testing P2P payment with inactive receiver wallet...
Wallet active status updated for roll number 2 (is_active: 0).
Warning: Wallet for receiver roll number 2 is inactive. Skipping P2P payment.
Wallet active status updated for roll number 2 (is_active: 1).


PL/SQL procedure successfully completed.
```

Conclusion

The "CampusPay" RFID-based unified Tap-and-Pay system effectively addresses the challenges of a fragmented payment ecosystem at Thapar Institute of Engineering and Technology by introducing a solution that utilizes existing ID cards and a centralized SQL database. This RFID-based unified Tap-and-Pay system enables students and staff to conduct seamless transactions across campus with a single platform, eliminating the need for multiple apps and reducing reliance on internet connectivity. Vendors benefit from simplified reconciliation and settlement processes, supported by accurate and timely financial reporting, while the system's security features—such as the "is_active" flag for wallet deactivation and centralized transaction logging—minimize fraud risks and ensure data integrity.

CampusPay's scalable and sustainable design positions it as a cost-effective model for future expansion, with potential integrations like advanced spending analytics and campus resource management tools. By leveraging existing RFID infrastructure and low-cost scanners, the RFID-based unified Tap-and-Pay system offers a practical solution that can serve as a blueprint for other educational institutions. This project not only enhances the daily experience of TIET's community but also underscores the institute's commitment to innovation, establishing a cashless, efficient, and secure payment environment that aligns with its forward-thinking ethos.

References

- Class notes and lab assignment
- https://dev.mysql.com/doc/
- https://www.w3schools.com/sql/
- https://learn.sparkfun.com/tutorials/rfid-basics/all
- https://www.nxp.com/docs/en/data-sheet/MF1S50YYX_V1.pdf